INFORME TÉCNICO DE CONSULTORÍA

Robustez de la Integridad de la Información en las Transmisiones (CAI 1 - Parte 2)

Asignatura: Seguridad en Sistemas de Información e Internet | **Equipo:** Security Team 2 (ST-2)
Autores: Raimundo García Pérez, Konstantinos Peridis, Luis Alarcón Conradi
Fecha: Septiembre de 2026 | **Código de Entrega:** ST-2-CAI1B

1. Resumen Ejecutivo

A requerimiento de la entidad bancaria, el equipo de auditoría de **SecurTech Advisory** ha evaluado la robustez del esquema de integridad empleado en las transferencias realizadas desde la aplicación móvil de clientes, basado en **Códigos de Autenticación de Mensajes (HMAC)** con claves secretas de **32 bits** distribuidas anualmente en un dispositivo físico (pendrive, smartcard o SD memory).

A partir de los tres mensajes de transferencia remitidos por el cliente, junto con sus HMAC codificados en Base64, el equipo auditor determinó que el algoritmo de generación empleado es **HMAC-SHA-256** (deducido de la longitud del resumen: 32 bytes / 256 bits) y ejecutó un ataque de **fuerza bruta exhaustiva** sobre el espacio completo de claves de 32 bits (2^{32} combinaciones), recuperando el **100 % de las claves secretas** en **9 minutos y 4 segundos**, utilizando hardware modesto (CPU de un portátil, sin GPU dedicada).

Dictamen Técnico Principal: el tamaño de clave de 32 bits empleado por la entidad es **manifiestamente inseguro**. Se recomienda su sustitución inmediata por claves de al menos **128 bits**, idealmente **256 bits** (igualando la longitud del resumen HMAC-SHA-256, conforme a las recomendaciones de RFC 2104 y NIST SP 800-107), así como una revisión integral del proceso de distribución y renovación de claves a los clientes.

2. Resumen Ejecutivo de la Metodología Aplicada

Para garantizar un proceso riguroso y reproducible, la auditoría se articuló en cuatro fases operativas consecutivas (el detalle completo, con comandos y capturas, se recoge en el Anexo IV):

- **Normalización Criptográfica:** los HMAC facilitados se encontraban en Base64 con una extensión de 44 caracteres (32 bytes decodificados / 256 bits), lo que permitió deducir de forma inequívoca el uso de HMAC-SHA-256 (modo Hashcat **-m 1450**).
- **Selección del Vector de Ataque:** al tratarse de una clave aleatoria distribuida por la propia entidad (no elegida por un usuario), se descartaron los ataques por diccionario en favor de una **fuerza bruta exhaustiva** (**-a 3**) con máscara **?b?b?b?b**, que recorre el rango completo de cada byte (0x00-0xFF) y agota exactamente el espacio de 2^{32} claves posibles.
- **Medición de Capacidad de Cómputo:** se realizó un benchmark limpio con hashcat -m 1450 -b sobre el hardware disponible (CPU Intel i7-1065G7, sin GPU), obteniendo una velocidad de referencia de **14.717,1 kH/s**.
- **Ejecución del Ataque y Verificación Cruzada:** se lanzó el ataque sobre los tres mensajes simultáneamente, recuperando las tres claves, y se verificó cada una de forma independiente recalculando el HMAC con OpenSSL.

3. Anexo del Proceso: Claves Obtenidas (Punto a)

Conforme solicita el cliente, se detalla a continuación el proceso seguido y las claves obtenidas, para su revisión por el Security Team.

3.1. Deducción del Algoritmo y Preparación de los Datos

Los tres HMAC recibidos decodifican de Base64 a exactamente 32 bytes, lo que identifica **HMAC-SHA-256**. Se generó el fichero hashes_hmac.txt en formato hash_hex:mensaje mediante el script **hmac_to_hashcat.py** (Anexo I):

Mensaje	HMAC (Base64)	HMAC (hex)
34567893 987344 120	CLxpGZUJ6GkQ...oIu2Yv5Zk=	08bc6919...662fe599
34567894 986454 10	we7bS6LiKkOd...OeQGeCfyI=	c1eedb4b...067827f22
34537895 957769 110	bT42fKu1Yvpr...YxIrRg3I=	6d3e367c...48ad18372

3.2. Ataque de Fuerza Bruta Exhaustiva

```
hashcat -m 1450 -b -D 1 -O                                  # benchmark: 14.717,1 kH/s
hashcat -m 1450 -a 3 -D 1 -O hashes_hmac.txt ?b?b?b?b \
        --outfile soluciones_hmac.txt
```

El ataque recuperó las tres claves en **9 minutos y 4 segundos** (93,66 % del keyspace recorrido), a una velocidad sostenida de ~12,6-13,9 MH/s en una CPU de portátil sin GPU:

Mensaje	Clave (hex)	Representación
34567893 987344 120	20216767	" !gg" (ASCII imprimible)
34567894 986454 10	99fa46a6	binario no imprimible
34537895 957769 110	deadbeef	binario (patrón "deadbeef")

Cada clave fue verificada de forma independiente recalculando el HMAC con OpenSSL (openssl dgst -sha256 -mac HMAC -macopt hexkey:<clave>), confirmando la coincidencia exacta con los valores recibidos en los tres casos. El proceso completo, con capturas de consola, se adjunta en el **Anexo IV** (metodologia_ejecucion.pdf) y las evidencias crudas en el **Anexo III** (carpeta evidencias/).

4. Respuesta a la Consulta (a): Seguridad del Tamaño de Clave

El cliente solicita informar sobre la seguridad del tamaño de clave de 32 bits y el tiempo necesario para encontrarla, razonando si dicho tiempo es suficientemente alto.

Escenario	Espacio recorrido	Velocidad	Tiempo estimado
Equipo auditor (caso medio)	2^{31}	14.717,1 kH/s	145,9 s (≈ 2,4 min)
Equipo auditor (peor caso)	2^{32}	14.717,1 kH/s	291,8 s (≈ 4,9 min)
Ejecución real (3 claves)	93,66 % de 2^{32}	≈ 12,6-13,9 MH/s	544 s (9 min 4 s)
Atacante ×1000 (caso medio)	2^{31}	14.717,1 MH/s	≈ 0,146 s

Conclusión: el tamaño de clave de 32 bits **no es seguro**. Con hardware ordinario (una CPU de portátil, sin GPU ni clúster de cómputo) el equipo auditor recuperó las tres claves reales del sistema en menos de 10 minutos, y el tiempo medio teórico para romper una única clave de 32 bits es de apenas 2,4 minutos. Un atacante con una capacidad de cómputo 1000 veces superior (perfectamente alcanzable con una GPU moderna, un clúster de cracking o un FPGA/ASIC dedicado) rompería la clave en **una fracción de segundo**

(≈146 ms de media). Dado que las claves protegen transferencias financieras activas durante todo un año, un tiempo de ruptura medido en minutos u horas es absolutamente insuficiente frente a la ventana de exposición real del sistema.

5. Respuesta a la Consulta (b): Recomendación de Tamaño de Clave y Proceso de Entrega

5.1. Tamaño de Clave Recomendado

Para razonar el tamaño mínimo recomendado se ha tomado como criterio la propia política de la entidad: la clave se distribuye **una vez al año** y debe permanecer válida durante todo ese periodo. Por tanto, el tamaño de clave debe resistir un ataque de fuerza bruta sostenido durante **1 año** incluso frente a un atacante con una capacidad de cómputo **1000 veces superior** a la del equipo auditor.

```
vel_atacante        = 14.717.100 H/s x 1000 = 14.717.100.000 H/s
combinaciones_1_anio = vel_atacante x (365,25 x 86.400) s ≈ 4,64 x 10^17
bits_minimos         = techo(log2(combinaciones_1_anio)) ≈ 59 bits
```

El cálculo anterior arroja un **mínimo matemático de ≈59 bits** para resistir un año completo de ataque frente a ese atacante concreto. No obstante, este valor es un límite inferior calculado sobre la capacidad de cómputo *actual* del atacante hipotético, y no incorpora margen de seguridad frente a mejoras tecnológicas futuras (ley de Moore, ASICs dedicados, computación en la nube a gran escala) ni frente a atacantes con muchos más recursos que el factor ×1000 considerado. Por ello, siguiendo el principio de *defensa en profundidad* y los estándares de la industria, se recomienda:

- **Mínimo aceptable: 128 bits.** Es el nivel de seguridad simétrica estándar recomendado por NIST (SP 800-57) para protección a largo plazo, y deja un margen de 2■■ respecto al mínimo calculado de 59 bits.
- **Recomendación óptima: 256 bits.** Iguala la longitud del resumen de HMAC-SHA-256, en línea con RFC 2104 y NIST SP 800-107, que recomiendan que la clave HMAC tenga una longitud igual o superior a la del resumen de la función hash subyacente para no degradar la seguridad teórica del algoritmo.

5.2. Mejora del Proceso de Entrega de Claves

Independientemente del tamaño de clave elegido, el equipo auditor detecta debilidades estructurales en el proceso actual de distribución (dispositivo físico entregado una vez al año, con una única clave estática usada durante todo el periodo):

- **Reducir la ventana de exposición:** sustituir la clave estática anual por un esquema de **renovación periódica** (p. ej. trimestral) o de **derivación de claves de sesión/transacción** (HKDF) a partir de un secreto maestro, de forma que el compromiso de una clave derivada no exponga todo el periodo de validez.
- **Almacenamiento seguro en el dispositivo:** emplear elementos seguros (*secure element*) o smartcards con protección por PIN/biometría en lugar de un pendrive o SD memory de propósito general, que puede copiarse sin control ni detección si se pierde o es sustraído.
- **Canal de entrega y activación:** incorporar un mecanismo de activación por canal independiente (p. ej. validación en la app tras verificación de identidad) para que la posesión física del dispositivo no sea, por sí sola, suficiente para operar la clave, y habilitar revocación/reemisión inmediata ante pérdida o sospecha de compromiso.

- **Protección de la integridad frente a repetición (replay):** un HMAC robusto protege el contenido del mensaje, pero no impide por sí solo que un atacante reenvíe un mensaje legítimo capturado previamente. Se recomienda incluir en el mensaje autenticado un **número de secuencia o marca temporal (nonce/timestamp)** verificado por el servidor, para mitigar los ataques de repetición y man-in-the-middle mencionados en la Política de Seguridad de la entidad.

6. Bibliografía

- Ceballos Guerrero, Rafael. *Tema 2: Integridad de la Información (Partes 1 y 2)*. Grupo de Investigación Quivir, Dpto. Lenguajes y Sistemas Informáticos, Universidad de Sevilla.
- RFC 2104 - HMAC: Keyed-Hashing for Message Authentication. IETF.
- NIST SP 800-107 Rev. 1 - Recommendation for Applications Using Approved Hash Algorithms.
- NIST SP 800-57 Part 1 Rev. 5 - Recommendation for Key Management.
- Documentación oficial de Hashcat (v7.x): modos de ataque, máscaras y catálogo de hash-modes: https://hashcat.net/wiki/.

7. Relación de Anexos Entregados

Junto al presente informe, el equipo auditor adjunta el siguiente paquete de evidencias digitales:

- **Anexo I: Script de Normalización HMAC (hmac_to_hashcat.py):** módulo Python 3 que decodifica los HMAC en Base64 recibidos, deduce el algoritmo (HMAC-SHA-256) y genera el fichero de entrada para Hashcat en formato modo 1450 (hash:mensaje).
- **Anexo II: Script de Análisis de Tiempos (analizar_hmac.py):** calcula los tiempos de ruptura medio/peor caso, la velocidad de un atacante ×1000 y los bits mínimos recomendados a partir de la velocidad de Hashcat medida.
- **Anexo III: Evidencias Digitales (carpeta evidencias/ y ficheros de datos):** hashes_hmac.txt (entrada Hashcat), soluciones_hmac.txt (salida cruda de Hashcat), soluciones.txt (registro consolidado de claves recuperadas), evidencias/benchmark.log y evidencias/analisis_tiempos.log.
- **Anexo IV: Memoria Metodológica Detallada (metodologia_ejecucion.pdf):** entorno de pruebas (Kali Linux, backend OpenCL de CPU), justificación del vector de ataque, comandos completos de Hashcat y verificación cruzada con OpenSSL.
- **Anexo V: Script de Automatización del Ataque (run_attack.sh):** script de shell que orquesta el benchmark y el ataque de fuerza bruta con la máscara ?b?b?b?b sobre hashes_hmac.txt.

Informe emitido en Sevilla por SecurTech Advisory para la Entidad Financiera. Todos los derechos reservados.